DBMS PROJECT REPORT

Chandigarh University Academic Opportunity Tracker

Opportunity Compass

Student and administrator portals with explainable eligibility and readiness

Chandigarh University
November 2026

Submission details

Student name _____

University UID _____

Section _____

Department _____

Faculty name _____

Project implementation

Python Flask and MySQL 8 with HTML CSS JavaScript, session authentication, student preparation workflows and administrator review. The selected application runs at http://127.0.0.1:5000.

Abstract

The Chandigarh University Academic Opportunity Tracker supports the student journey from finding an opportunity to preparing and tracking an application. Students maintain academic and skill profiles, search current listings, inspect eligibility explanations, save opportunities, manage document readiness and preparation tasks, submit applications and follow status history. Administrators publish and update opportunities, inspect applicants and record review decisions.

The system uses Flask for server-side routes, Jinja for escaped templates and MySQL for relational persistence. It reuses the existing normalized schema and adds an Under Review application state through a separate migration. Related opportunity requirements and application status changes are saved transactionally. A unique student-opportunity pair prevents duplicate applications.

Eligibility checks academic profile, allowed department, CGPA, year, listing status and deadline. Readiness is a transparent weighted calculation: 40 percent academic rules, 30 percent required skills, 20 percent mandatory documents and 10 percent preparation tasks. It is not a machine learning model and does not guarantee selection.

The current verification evidence records 19 passing automated test methods and 33 successful authenticated HTTP page checks, including all 21 MySQL schema views. Security measures include CSRF tokens, current role and ownership validation, hashed passwords, revocable sessions, rate limits, parameterized SQL and restricted database grants. Demonstration records are explicitly fictional, and screenshots must use these records rather than personal student data.

Scope and boundaries

This report covers the Documents CampusOpportunityPlanner application. Document preparation is a self-reported checklist; no file uploads or independent document verification are implemented. Academic information and skills are self-reported. Production TLS, external identity verification and backup operations remain deployment responsibilities.

Contents

The report follows the implementation from requirements through architecture, student workflows, administrator workflows, algorithms, database design, security, testing, operation and references. The final three ER sheets show identity, opportunity and application relationships with PK, FK and unique markers.

Chapter	Subject
1	Problem statement and requirements
2	Architecture and code responsibilities
3	Student portal implementation
4	Administrator portal implementation
5	Eligibility readiness and status algorithms
6	Relational design and normalization
7 to 9	Data dictionary for all 21 tables
10	Security controls and integrity
11	Testing and verified results
12	Operation limitations and conclusion
References	Formatted technical references
ER sheets	Identity opportunities and applications

Problem statement and requirements

Campus opportunities often have different academic conditions, skills, documents and closing dates. A simple listing page does not tell a student why they qualify or what preparation remains. The project combines discovery, understandable rules, personal preparation and application review in one relational application.

Objectives

The system must connect each action to an authenticated student or administrator, calculate guidance from stored records, preserve application history and prevent duplicate or unauthorized changes. It must also make the next action clear: complete a profile, build a required skill, prepare a document, finish a task, submit or wait for review.

Requirement	Implemented behavior
Student dashboard	Database counts recommendations deadlines and recent status updates
Search and filters	Keyword category department skill mode deadline eligible-only and sorting
Eligibility and readiness	Server-side explanations component scores gaps and next action
Save and apply	One application record per student and opportunity; saved records can be submitted
Application tracking	Submission date status history withdrawal documents and tasks
Admin portal	Role redirect overview opportunity editing applicant filtering and review
Database evidence	Authenticated schema browser for 21 tables without exposing row secrets

Technology

The reviewed runtime uses Flask 3.1.3, MySQL Server 8.0.46, MySQL Connector Python 26.10.0, Werkzeug 3.1.9, python dotenv 1.2.4 and Waitress 3.0.2. Flask provides the web framework and server-rendered application conventions [1]. MySQL provides transactions and integrity constraints [2].

Architecture and code responsibilities

A browser submits forms to Flask, which authenticates and authorizes the request before reading or updating MySQL. Templates display escaped results. The browser never supplies authoritative eligibility, readiness, account role, owner ID or application state.

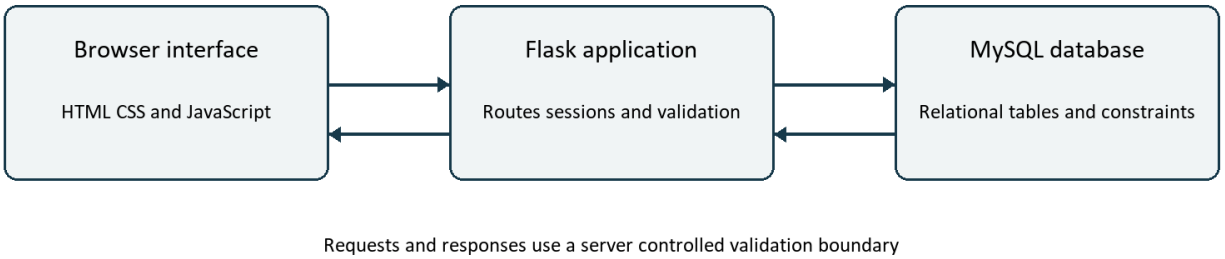


Figure 1 Web application architecture

File or module	Responsibility
app.py	Existing registration login setup logout and role-based landing redirects
security.py	CSRF account checks revocable sessions limits headers and safe errors
database.py	Prepared values query helpers request connection cleanup and transactions
student_portal.py	Dashboard search opportunity save/apply profile preparation and ownership
admin_portal.py	Overview opportunity transactions review transitions and schema browser
portal_services.py	Batched database enrichment eligibility explanations and readiness calculation

The migration script grants only the DML operations required by implemented routes. MySQL metadata is read through INFORMATION_SCHEMA for the schema browser and ER export. No environment values, password hashes or session tokens are rendered in that browser.

Student portal implementation

Dashboard and discovery

After login, a student reaches the dashboard. It displays profile completion, active submitted applications, saved opportunities, upcoming deadlines, recommendations and recent application updates. Recommendations are ordered by eligibility and readiness. A student without an academic profile receives a setup action rather than an invented eligibility result.

Discovery lists published opportunities whose deadlines remain open. It supports keyword, category, department, skill, mode, closing-date and eligible-only filters. Sorting uses fixed deadline, newest or readiness choices. Each card shows organization, category, academic requirements, deadline, eligibility, readiness and the student application state.

Opportunity and preparation

The detail page presents the full description, department restrictions, academic limits, required or preferred skills, documents and explainable score. Saving creates a unique application in the saved state without claiming it is submitted. The student can then mark document readiness and add dated tasks. Completing or reopening a task changes the corresponding preparation component.

The profile page updates academic details and self-reported skill levels. Skill matching checks whether a required skill is present; the schema does not define minimum skill proficiency thresholds. Required documents are ready only when their application checklist state is ready, submitted or approved.

Applications and status

An eligible student may submit directly or submit a saved record. The backend locks the opportunity, rechecks its current status and deadline, then records applied status and submission time with history. A repeated submission does not create another record. My Applications combines saved and submitted records and links to detailed history, documents and tasks. Students can withdraw applied, under-review or shortlisted applications; selected, rejected and withdrawn states are final.

Route group	Student action
/student/dashboard	Personal overview
/student/discover	Search and filters
/student/opportunities/id	Details save and apply
/student/applications	Saved and submitted list
/student/applications/id	History checklist tasks and withdrawal
/student/profile	Academic and skill updates

Administrator portal implementation

Authentication and overview

Students and administrators use the same secure sign-in form. The backend redirects administrators to their own dashboard using the database role. Student routes reject administrators and admin routes reject students. The overview uses real SQL counts for active students, open opportunities, submitted applications and pending reviews, with category and status distributions.

Opportunity management

Administrators create or edit organization/category assignment, title, description, mode, location, deadline, optional dates, stipend and external information URL. They define year/CGPA limits, allowed departments, required/preferred skills and mandatory documents. Listing states include draft, published, closed and cancelled. Closing or cancelling retains applications.

The main listing and its academic and junction-table requirements save in one transaction. Every foreign selection must exist. Dates, numeric limits, http or https URLs and mutually exclusive required/preferred skills are checked server-side. The opportunity row also provides the serialization boundary against a concurrent application.

Applicant review

The applicant list supports search and status filtering. Review shows the applicant profile, skill evidence, current readiness, document checklist and audit trail. Allowed transitions are submitted to Under Review, Shortlisted or Rejected; Under Review to Shortlisted, Selected or Rejected; and Shortlisted to Selected or Rejected. Saved/preparing and final states cannot receive review decisions.

A review transaction locks the application and atomically updates status, inserts an immutable history record and creates a student notification. Internal notes are visible only to administrators. Current scores may change when profile or opportunity requirements change; they are not immutable submission snapshots.

Database schema browser

The admin-only schema browser shows live column names, types, nullability, primary/unique markers and foreign-key targets for all 21 tables. It is a metadata interface, not a display of passwords, token values or real student records. It supplies a safe view for MySQL structure evidence.

Eligibility readiness and status algorithms

Academic and availability rules

The eligibility service checks for a usable academic profile, allowed department when restricted, minimum CGPA when greater than zero, year within the configured bounds, published listing state and a future application deadline. Every rule produces a true/false explanation. An absent condition is not replaced with a guessed value.

Transparent readiness formula

Component	Weight	Calculation
Academic eligibility	40	Fraction of applicable academic checks satisfied
Required skills	30	Fraction of required skill IDs present in student skills
Mandatory documents	20	Fraction of mandatory checklist documents ready
Preparation tasks	10	Fraction of linked tasks completed

Total readiness is the sum of weighted components, rounded to a percentage. A component with no requirements receives full credit. Preferred skills do not reduce the score. Listing availability is a separate eligibility gate: a closed opportunity can still have a high preparation score but cannot accept an application. Missing skills/documents guide preparation; academic eligibility and availability control submission.

Worked example

A student who satisfies all academic rules, has one of two required skills, one of two mandatory documents ready, and one of two tasks complete receives 40 + 15 + 10 + 5 = 70 percent. A student with all required evidence and completed tasks receives 100 percent. The result represents recorded preparation, not a selection probability.

Application lifecycle

Save creates saved and an initial history record. Apply changes saved/preparing to applied or creates applied directly, with applied_at set. Admin review follows the allowed transitions above. Withdrawal is permitted only before a final outcome. Unique(student_id, opportunity_id) prevents duplicate records even after withdrawal. Each history entry records actor, old/new status and time.

Relational design and normalization

The live database contains 19 original domain tables and two security tables. Identity, academics, reference data, requirements and operational records are separated. Junction tables represent repeated relationships without comma-separated lists. Foreign keys connect existing parents and children; unique keys constrain identity and duplicate relationships [2].

Relationship	Cardinality
users to student_profiles	One to zero or one; user_id unique in profile
departments to student_profiles	One to zero or many
student_profiles to skills	Many to many through student_skills
organizations or categories to opportunities	One to zero or many
opportunities to eligibility_requirements	One to zero or one
students to opportunities	Many to many through applications
applications to status history tasks documents	One to zero or many
users to auth_sessions	One to zero or many

Normalization

Atomic columns and separate relationship rows support first normal form. In composite-key junctions, proficiency, requirement type and document state describe the complete pair, supporting second normal form. Department, organization, category, skill and document descriptions are stored once in reference tables rather than copied into applications. This limits transitive duplication and follows the intent of third normal form.

Migrations and integrity

The first security migration adds sessions, rate buckets and rejecting history update/delete triggers. The portal migration adds under_review at the end of the application enum, preserving existing labels. No table is dropped and no existing record is reset. Foreign-key cascades remain; a privileged parent deletion can bypass child history triggers, so future deletion policies need separate retention review. The runtime has no user/application delete permission.

Student and identity data dictionary

users

user_id [PK]; full_name; email [UQ]; password_hash; role; is_active; created_at.

student profiles

student_id [PK]; user_id [UQ FK]; university_uid [UQ]; department_id [FK]; current_year; cgpa; graduation_year.

departments

department_id [PK]; department_code [UQ]; department_name [UQ].

skills

skill_id [PK]; skill_name [UQ]; skill_category.

student skills

student_id [PK FK]; skill_id [PK FK]; proficiency_level; is_verified.

student interests

student_id [PK FK]; category_id [PK FK]; interest_level.

PK identifies a primary key, FK a foreign key, and UQ a separately unique field. Composite keys are shown by marking each participating primary-key column. The schema metadata and ER figures are generated from the live MySQL definition.

Opportunity and application data dictionary

organizations

organization_id [PK]; organization_name [UQ]; organization_type; website_url; contact_email; created_at.

opportunity categories

category_id [PK]; category_name [UQ]; description.

opportunities

opportunity_id [PK]; organization_id [FK]; category_id [FK]; title; description; mode; location; stipend; application_url; application_deadline; start_date; end_date; is_campus_managed; listing_status; created_by [FK]; created_at; updated_at.

eligibility requirements

opportunity_id [PK FK]; minimum_cgpa; minimum_year; maximum_year; all_departments_allowed.

opportunity departments

opportunity_id [PK FK]; department_id [PK FK].

opportunity skills

opportunity_id [PK FK]; skill_id [PK FK]; requirement_type.

applications

application_id [PK]; student_id [FK]; opportunity_id [FK]; application_status; applied_at; external_reference; created_at; updated_at.

application status history

history_id [PK]; application_id [FK]; old_status; new_status; changed_by [FK]; change_note; changed_at.

PK identifies a primary key, FK a foreign key, and UQ a separately unique field. Composite keys are shown by marking each participating primary-key column. The schema metadata and ER figures are generated from the live MySQL definition.

Document preparation and security dictionary

document types

document_type_id [PK]; document_name [UQ]; description.

opportunity documents

opportunity_id [PK FK]; document_type_id [PK FK]; is_mandatory.

application documents

application_id [PK FK]; document_type_id [PK FK]; document_status; file_name; notes; updated_at.

preparation tasks

task_id [PK]; application_id [FK]; task_title; due_date; task_status; completed_at; created_at.

notifications

notification_id [PK]; user_id [FK]; notification_type; title; message; opportunity_id [FK]; application_id [FK]; is_read; created_at.

auth sessions

session_hash [PK]; user_id [FK]; created_at; expires_at; revoked_at.

security rate limits

bucket_key [PK]; window_start; hits.

PK identifies a primary key, FK a foreign key, and UQ a separately unique field. Composite keys are shown by marking each participating primary-key column. The schema metadata and ER figures are generated from the live MySQL definition.

Security controls and integrity

Control	Implemented behavior
Authentication	CU UID/email consistency, password length and Werkzeug hashing with generic login errors [3]
Sessions	Opaque token with server-side digest and absolute expiry; logout revocation and current account check
Authorization	Reusable roles plus student-scoped application/document/task/profile queries
CSRF	Every POST form has a random session token; invalid/missing tokens rejected before action
SQL and transactions	Parameterized request values, allowlisted identifiers, rollback and cursor/connection cleanup
Rate limits	Login IP/account, registration IP, student apply/save and admin review with MySQL atomic counters
Browser and errors	CSP, frame denial, no-sniff, referrer and permissions policy, safe errors and no-store
Database grants	Dedicated non-root runtime, no DDL, no history mutation and narrowly scoped table rights

Login allows 30 attempts/minute per direct IP and 8 per normalized account; registration allows 20/hour per direct IP. Save/apply allows 30/minute per student and review 60/minute per admin. Limits are shared across workers, return Retry-After and ignore untrusted forwarding headers.

No file upload is claimed. Document readiness and skills are self-reported. Local private files are ignored and access-restricted, but production must move migration credentials outside the runtime-readable identity. The audit identified no configured-secret exposure in scanned project text. A passing scan or test does not establish that every infrastructure or unknown vulnerability is absent.

Testing and verified results

The current full run passed 19 automated test methods with MySQL tests enabled and zero skips. The suite includes the original 16 security tests, a complete portal workflow test, a readiness rule test and a portal mutation rate-limit test. Nested assertions exercise multiple conditions within each method. The standard unittest framework organizes the automated checks [5].

Verification	Evidence	Result
Student and admin login redirects	Role-specific dashboard and cross-role 403	Pass
Opportunity management	Create/edit listing and all related requirements	Pass
Discovery	Keyword category department skill mode eligible and score sort	Pass
Readiness	Exact weights, gaps, documents/tasks and 100-percent completion	Pass
Save and apply	Saved-to-applied transition and duplicate prevention	Pass
Ownership	Another student denied application document task and withdrawal actions	Pass
Review and history	Concurrent requests produce one transition; private notes withheld	Pass
Notifications and withdrawal	Atomic student notification and terminal-state rules	Pass
Security regression	CSRF injection escaping sessions input errors and grants	Pass
Abuse control	Apply and review limits return 429/Retry-After	Pass

A separate live HTTP run made 33 successful authenticated GET checks: six student pages, six admin pages and 21 MySQL schema views. These are page checks, not 33 distinct user workflows. JSON records retain path, role and status without cookies or passwords.

The recorded pip-audit package scan found zero known advisories and zero skipped packages; pip check found no broken requirements [4]. The current source and live schema are the evidence for CSRF, rate limiting and restricted access claims. Raw sanitized outputs remain with the project for review.

Operation limitations and conclusion

Run and verify

Use the Documents CampusOpportunityPlanner directory and launch start.ps1. The local URL is http://127.0.0.1:5000. Apply scripts/setup_portal.py with private migration credentials when moving an existing audited database to the portal version. The schema name remains campus_opportunity_planner.

.\.venv\Scripts\waitress-serve.exe --listen=127.0.0.1:5000 app:app

Tests: set RUN_MYSQL_TESTS=1 and run python -m unittest discover -s tests -v. Run scripts/portal_live_checks.py for current live page evidence. Use generated fixtures only; do not run fixture tests on a production dataset.

Demonstration data

Eight opportunities, fictional organizations and dedicated demonstration identities support review. Their descriptions explicitly state they are not real openings. Report screenshots should use these accounts. Demonstration passwords stay only in ignored local environment configuration and are not included in this report.

Remaining limitations

No upload pipeline, email ownership verification, university SSO, password recovery or MFA is implemented. Academic values and skills are self-reported. No external decision should treat readiness as verified academic evidence or an automated selection probability. Production TLS, proxy trust, remote-database TLS, backups, retention and independent audit storage require deployment review.

The original landing-page readiness card is labelled as illustrative. Student and admin dashboards use real database queries. Current readiness recalculates from current data and is not a stored historical submission snapshot.

Conclusion

The implemented system now supports the main student and administrator opportunity-tracking cycle rather than only account setup. Students discover, evaluate, save, prepare, apply and track; administrators manage requirements and review applicants. A normalized MySQL schema, explicit rules and testable transaction/authorization controls provide the basis for further production work.

References

[1] Pallets Projects. (n.d.). *Flask documentation Version 3.1.* Retrieved November 3, 2026. Online documentation

[2] Oracle. (n.d.). *MySQL 8.0 Reference Manual.* Retrieved November 3, 2026. Online documentation

[3] Pallets Projects. (n.d.). *Werkzeug utilities and security helpers.* Retrieved November 3, 2026. Online documentation

[4] Python Packaging Authority. (n.d.). *pip audit project documentation.* Retrieved November 3, 2026. Online documentation

[5] Python Software Foundation. (n.d.). *unittest Unit testing framework.* Retrieved November 3, 2026. Online documentation

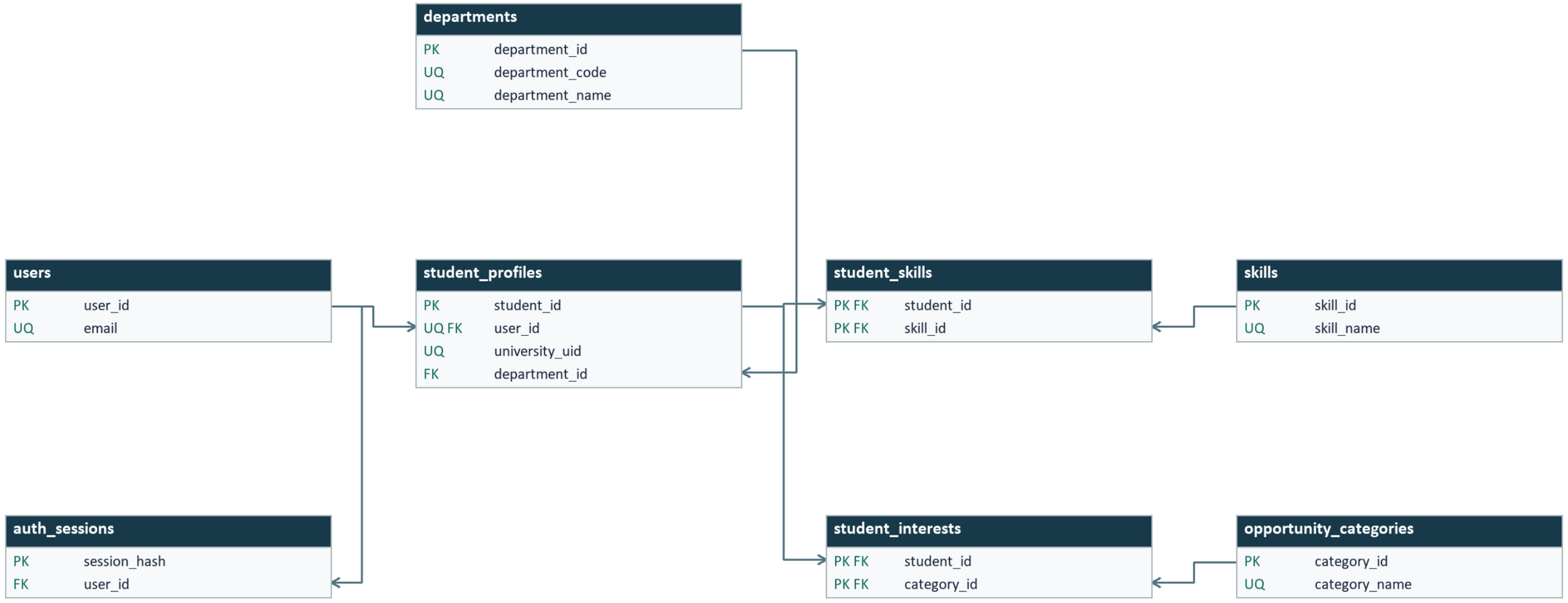
Project evidence

Implementation: app.py, student_portal.py, admin_portal.py, portal_services.py, database.py and security.py. Schema: database/schema.sql and separate migrations. Verification: tests/test_security.py, tests/test_portal.py, security-audit/portal-test-results.txt and security-audit/portal-http-results.json. ER export: reports/schema_metadata.json contains schema metadata only, not personal record values.

ER diagram for student identity

Academic Opportunity Tracker ER diagram

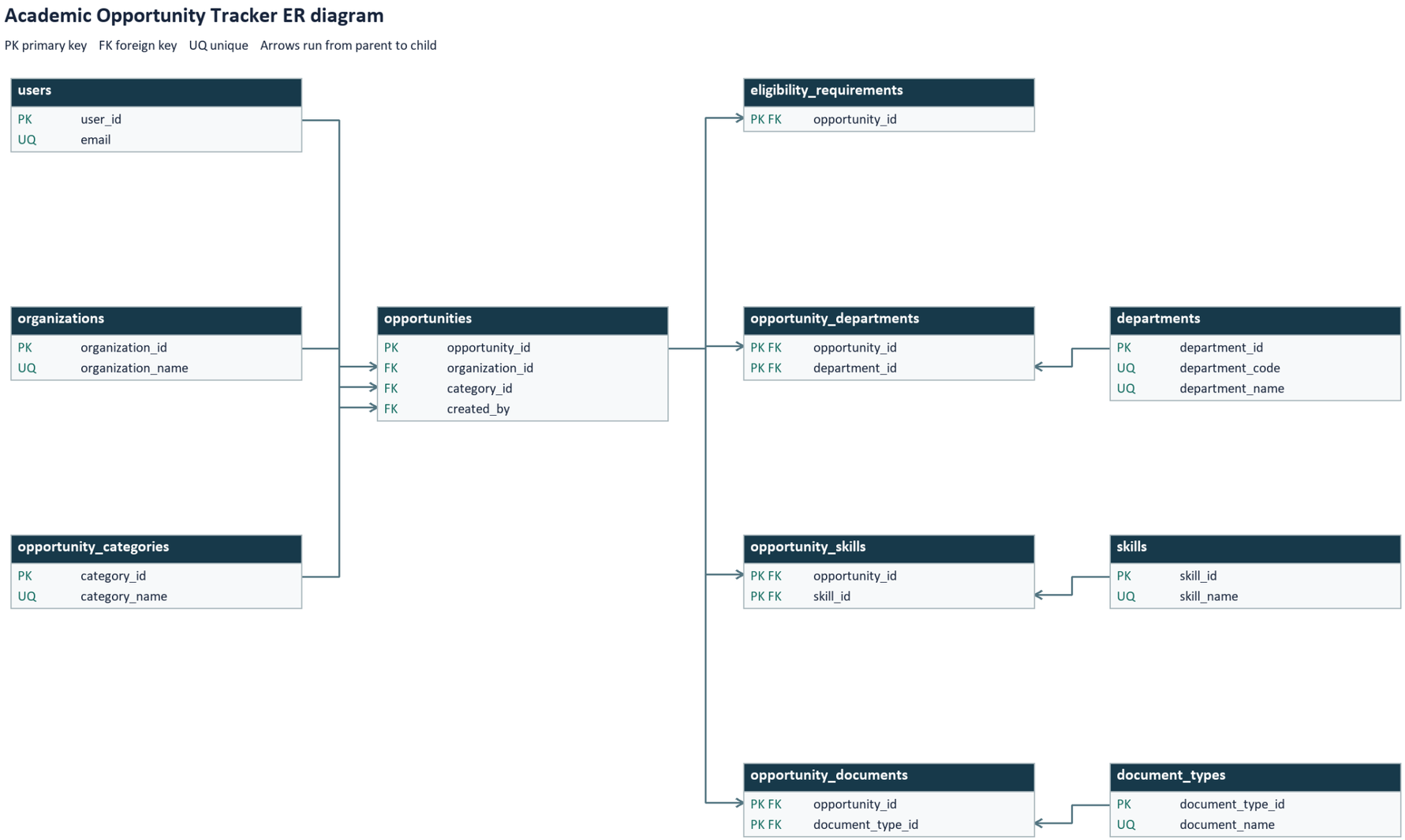
PK primary key FK foreign key UQ unique Arrows run from parent to child



Cardinality: each parent has 0..N child rows; users to student_profiles and opportunities to eligibility_requirements are 0..1.
Nullable foreign keys allow a child to have no parent. security_rate_limits is an independent HMAC-keyed security table.

Live MySQL metadata defines the PK FK UQ markers and directed parent-to-child relationships. A separate er_complete.svg preserves all 21 table structures for zoomed inspection. Schema table screenshots are a distinct evidence item.

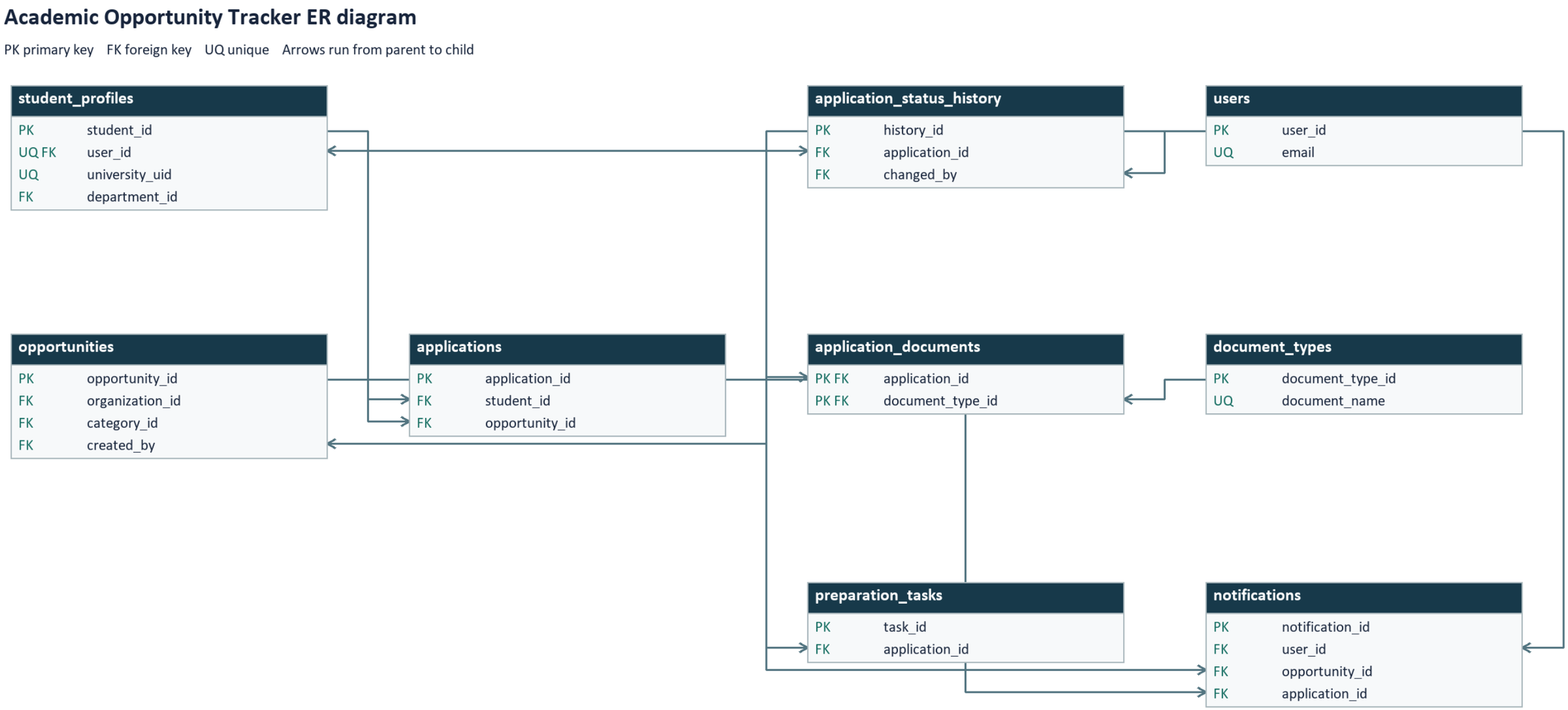
ER diagram for opportunity requirements



Cardinality: each parent has 0..N child rows; users to student_profiles and opportunities to eligibility_requirements are 0..1.
Nullable foreign keys allow a child to have no parent. security_rate_limits is an independent HMAC-keyed security table.

Live MySQL metadata defines the PK FK UQ markers and directed parent-to-child relationships. A separate er_complete.svg preserves all 21 table structures for zoomed inspection. Schema table screenshots are a distinct evidence item.

ER diagram for applications and review



Cardinality: each parent has 0..N child rows; users to student_profiles and opportunities to eligibility_requirements are 0..1.
Nullable foreign keys allow a child to have no parent. security_rate_limits is an independent HMAC-keyed security table.

Live MySQL metadata defines the PK FK UQ markers and directed parent-to-child relationships. A separate er_complete.svg preserves all 21 table structures for zoomed inspection. Schema table screenshots are a distinct evidence item.